DevHub Solo Production PRD

Document Control

Field	Value
Product name	DevHub — AI Developer Operating System
Document version	v3.0 Solo Production PRD
Status	Approved for implementation
Product owner	Ravi Shankar
Delivery model	Solo developer build
Product type	AI-powered developer workspace and productivity platform
Initial release goal	Web-first MVP with AI-assisted developer workflow
Architecture style	Modular monolith, production-ready, solo-maintainable

Product Definition

DevHub is a web-first AI-powered developer operating system that centralizes project management, learning progress, notes, goals, career preparation, resume handling, job tracking, and contextual AI assistance in one platform.[cite:1][cite:6] The product is intended to reduce fragmentation across tools such as note apps, coding platforms, calendars, resume sites, and generic AI tools by providing one connected workspace with an intelligence layer on top.[cite:1][cite:6]

The implementation strategy in this PRD is explicitly optimized for a solo developer. Architecture, scope, tooling, and roadmap choices are intentionally biased toward fast iteration, maintainability, low operational complexity, and future extensibility rather than multi-team parallel development.[cite:11][cite:14][cite:10]

Solo Developer Constraint

This product is initially being designed and built by a solo developer, so the system must avoid unnecessary infrastructure, over-engineered service decomposition, and broad first-release scope.[cite:11][cite:14] Every decision in this PRD is therefore finalized around a solo-friendly operating model: one codebase per tier, one primary database, one storage provider, one AI provider, one queue strategy, one deployment path, and one clearly phased roadmap.[cite:11][cite:14]

Mission

The mission of DevHub is to help developers become better engineers through AI-assisted planning, execution, learning, and career preparation.[cite:1][cite:6] The product should not just record activity; it should convert activity and user context into useful next actions.[cite:1][cite:6]

Product Positioning

DevHub should be positioned as an AI Developer Operating System rather than a developer dashboard because the product's value comes from connected workflows and contextual intelligence, not just a visual aggregation of widgets.[cite:1][cite:6] This framing also gives future AI modules a stable conceptual home without changing the product identity later.[cite:1]

Final Product Principles

- AI augments the user and keeps them in control.[cite:6]
- Each feature must map to a real developer workflow rather than vanity metrics.[cite:6]
- The MVP must feel coherent before it feels broad.[cite:11][cite:14]
- The system must stay modular enough to grow, but simple enough for one developer to maintain.[cite:11][cite:14]
- Data should become actionable through recommendations, summaries, and guided workflows.[cite:1][cite:6]

Target Users

Primary Users

- Computer Science students.[cite:1][cite:6]
- Placement aspirants.[cite:1][cite:6]
- Self-taught developers.[cite:1][cite:6]
- Early-career developers building projects and portfolios.[cite:6]

Secondary Users

- Freelancers managing skill growth and applications.[cite:6]
- Software engineers using the platform for side growth and interview preparation.[cite:6]

Core User Problem

Developers currently manage projects, progress tracking, notes, resumes, job applications, schedules, and learning resources across disconnected tools.[cite:1][cite:6] This produces fragmented context, weak prioritization, and poor visibility into what should be done next for real improvement.[cite:1][cite:6]

Product Promise

DevHub should know what the user is building, what they are learning, what jobs they are targeting, where they are weak, and what they should work on next.[cite:1][cite:6] The product wins if it makes a developer feel organized, guided, and technically sharper inside one workspace.[cite:6]

Final MVP Scope

The MVP scope is intentionally narrowed so a solo developer can ship a polished and production-ready first release.[cite:11][cite:14] The MVP shall include only the modules required to validate the core loop of planning, building, learning, and improving.[cite:11][cite:14]

MVP Modules

1. Authentication and user profile.
2. Dashboard.
3. Project manager.
4. Tasks and milestones.
5. Goals and habits.
6. Notes and document uploads.
7. Learning tracker.
8. Resume manager.
9. Job tracker.
10. Calendar and reminders.
11. AI assistant.
12. AI daily brief.
13. Basic AI resume review.
14. Basic GitHub integration.
15. Semantic search over user notes and uploaded docs.[cite:1][cite:6]

Out of Scope for MVP

The following items are explicitly excluded from the MVP to protect delivery quality and timeline:

- Multi-agent orchestration.[cite:1][cite:6]
- AI mock interviewer.[cite:1][cite:6]
- Advanced repository code review across full codebases.[cite:1][cite:6]
- Deep competitive coding platform integrations beyond future planning.[cite:1]
- Slack, Discord, email, Kaggle, GitLab, and Codeforces integrations.[cite:1]

- Team collaboration features such as shared workspaces, org roles, or collaborative project boards.[cite:6]
- Native mobile apps in the first release.[cite:11][cite:14]

Release Goal

The release goal is a stable web application that allows a solo user to manage their developer journey in one place and receive useful AI-generated assistance from their own data and goals.[cite:1][cite:6] The MVP should be production-ready in reliability and architecture, but disciplined in feature breadth.[cite:11][cite:14]

Final Tech Stack Decisions

This section freezes the implementation stack so development can start without ambiguity.

Frontend

- React
- TypeScript
- Vite
- TailwindCSS
- shadcn/ui
- React Router
- TanStack Query
- React Hook Form
- Zod[cite:1]

Backend

- Spring Boot
- Spring Security
- JWT authentication
- OAuth2 for GitHub login and integration
- Spring Data JPA
- Hibernate
- Flyway
- Redis only where necessary for caching and short-lived job state[cite:1]

Database

- PostgreSQL as the primary relational database
- pgvector extension for embeddings and semantic retrieval[cite:1]

Storage

- Supabase Storage as the file storage provider[cite:1]

AI

- OpenAI API as the primary model provider
- Spring AI as the integration layer
- Embeddings stored in PostgreSQL with pgvector[cite:1]

Background Jobs

- Spring Boot scheduled jobs for periodic tasks
- Database-backed job table for async AI and sync workflows
- No external message broker in MVP[cite:11][cite:14]

Deployment

- Frontend on Vercel
- Backend on Railway
- PostgreSQL on Neon
- Storage on Supabase Storage[cite:1]

Observability

- Application logs
- Spring Boot Actuator
- Basic monitoring dashboards later in production hardening phase[cite:1]

Tech Stack Rationale

The chosen stack preserves the spirit of the original brief while reducing solo-maintenance burden. A modular Spring Boot backend plus a React frontend is strong enough for production, while avoiding multi-service sprawl in the first version.[cite:1][cite:11][cite:14]

Redis remains optional and narrow in responsibility rather than becoming a foundational dependency for everything. This keeps the architecture easier to debug and deploy as a solo developer.[cite:1][cite:11]

Final Architecture Decision

DevHub will be built as a modular monolith. The frontend and backend are separate deployable applications, but the backend remains one production service with strongly separated internal modules.[cite:11][cite:14]

Why This Is Final

A modular monolith is the correct choice for a solo developer because it avoids the operational cost of microservices while preserving clean domain boundaries and future extraction paths.[cite:11][cite:14] This approach matches the user's consistent preference for locking architecture, schema, and module boundaries before feature coding begins.[cite:10][cite:11]

System Architecture

High-Level Flow

React frontend communicates with the Spring Boot backend through REST APIs.[cite:1] The backend writes business data to PostgreSQL, stores files in Supabase Storage, stores embeddings in PostgreSQL with pgvector, and calls OpenAI through Spring AI for scoped AI workflows.[cite:1]

Runtime Components

- Web client.
- API server.
- Relational database.
- File storage.
- Embedding store.
- Scheduled jobs.
- AI service layer inside backend modules.[cite:1][cite:6]

Final Backend Module List

The backend shall contain the following modules and no additional product modules in the initial repo:

1. `auth`
2. `users`
3. `dashboard`
4. `projects`
5. `tasks`
6. `goals`
7. `notes`

8. `learning`
9. `resumes`
10. `careers`
11. `calendar`
12. `integrations`
13. `knowledge`
14. `ai`
15. `notifications`
16. `audit`
17. `common`[cite:11][cite:7]

Final Frontend Module List

The frontend shall contain the following feature structure:

- `src/app`
- `src/shared`
- `src/features/auth`
- `src/features/dashboard`
- `src/features/projects`
- `src/features/tasks`
- `src/features/goals`
- `src/features/notes`
- `src/features/learning`
- `src/features/resumes`
- `src/features/careers`
- `src/features/calendar`
- `src/features/integrations`
- `src/features/knowledge`
- `src/features/ai`[cite:11][cite:7]

Final User Roles

Only two roles shall exist in MVP:

- `USER`
- `ADMIN`[cite:6]

The `ADMIN` role is reserved for internal maintenance and moderation needs, not for organization-level collaboration. There will be no workspace sharing or multi-user

collaboration in MVP.[cite:6]

Functional Modules

Authentication and User Profile

The system shall support email/password sign-up, login, logout, JWT session flow, and GitHub OAuth login.[cite:1] Users shall have one profile containing education details, skills, target roles, links, settings, preferences, and AI personalization metadata.[cite:1][cite:6]

Acceptance Criteria

- User can register and login securely.
- Protected routes require valid authentication.
- GitHub OAuth login works for supported accounts.
- User can edit profile and save preferences.[cite:1][cite:6]

Dashboard

The dashboard shall be the user's main landing surface after login.[cite:1] It shall present daily goals, project progress, learning progress, upcoming deadlines, brief insights, AI suggestions, and basic GitHub activity when connected.[cite:1][cite:6]

Acceptance Criteria

- Dashboard loads personalized data from all enabled modules.
- Empty state is supported for new users.
- Widgets degrade gracefully when integrations are not connected.[cite:1][cite:6]

Projects, Tasks, and Milestones

The project system shall allow users to create software projects with descriptions, stack tags, links, statuses, tasks, milestones, notes, and roadmap fields.[cite:1] The task system shall support kanban status, due dates, priority, and project linkage.[cite:1]

Acceptance Criteria

- User can create, edit, archive, and delete projects.
- User can create tasks and link them to projects.
- Milestone progress reflects task completion correctly.
- Project detail page supports documentation and notes.[cite:1]

Goals and Habits

Users shall define daily, weekly, monthly, and career goals plus optional habits with streak logic.[cite:1] Goals may optionally relate to projects, learning plans, or job preparation flows.[cite:1][cite:6]

Acceptance Criteria

- User can create and update goals.
- Habit streaks calculate correctly.
- Goal progress is visible on the dashboard.[cite:1]

Notes and Documents

The notes system shall support markdown notes, folders, tags, search, and document upload for personal knowledge indexing.[cite:1] Uploaded docs should be private per user and track extraction and indexing status.[cite:1][cite:6]

Acceptance Criteria

- User can create markdown notes.
- User can upload approved files.
- Search works for note titles, tags, and body content.
- Indexed documents are available for semantic retrieval after processing succeeds.[cite:1][cite:6]

Learning Tracker

The learning tracker shall support learning resources such as courses, books, tutorials, certificates, and videos, along with progress, estimated completion, and skill tags.[cite:1] It should act as a structured study tracker rather than a content platform.[cite:1][cite:6]

Acceptance Criteria

- User can add and manage learning resources.
- User can update progress and completion state.
- Dashboard reflects learning progress summaries.[cite:1]

Resume Manager

The resume manager shall support PDF upload, multiple named versions, metadata tracking, and AI-powered feedback outputs.[cite:1] The system will not attempt perfect resume parsing in v1; instead it should prioritize reliable text extraction and review quality.[cite:1][cite:6]

Acceptance Criteria

- User can upload multiple resumes.
- User can assign labels such as “backend,” “frontend,” or “general.”
- AI review produces score, issues, and suggestions.[cite:1][cite:6]

Job Tracker

The job tracker shall support companies, roles, application stages, deadlines, notes, and linked resume versions.[cite:1] Its purpose is tracking and preparation, not job scraping.[cite:1][cite:6]

Acceptance Criteria

- User can create and update applications.
- User can attach notes and status history.
- User can link a resume version and target role.[cite:1]

Calendar

The calendar shall support events for interviews, tasks, study blocks, deadlines, and meetings.[cite:1] Google Calendar sync is deferred from MVP, so this module must still feel useful as a standalone in-app calendar.[cite:1][cite:6]

Acceptance Criteria

- User can create, update, and delete events.
- Upcoming events appear on dashboard widgets and reminders.[cite:1]

GitHub Integration

GitHub integration shall support account connection, lightweight repository sync, contribution summaries, and basic profile analytics.[cite:1] The MVP will not perform full source-level code understanding across arbitrary repos.[cite:1][cite:6]

Final Scope

- GitHub OAuth connection.
- Import linked repositories.
- Store repository metadata.
- Show language usage and simple contribution insights.
- Manual resync and scheduled sync.[cite:1][cite:6]

AI Assistant

The AI assistant is part of MVP, but it is deliberately scoped. It shall answer questions, summarize user notes or documents, suggest next steps, and help with project planning, roadmap thinking, and resume or career guidance based on available user context.[cite:1] [cite:6]

Final Constraints

- Assistant context must be explicitly scoped per request.
- Responses should use user data only when relevant and permitted.
- There will be no multi-agent orchestration in MVP.[cite:6]

AI Daily Brief

The daily brief shall generate a concise daily summary using current tasks, goals, deadlines, learning items, and calendar events.[cite:1] This feature should combine rule-based prioritization with light AI summarization rather than heavy autonomous planning.[cite:1] [cite:6]

AI Resume Review

The first AI analysis workflow to ship shall be resume review because it has high visible value and bounded input size.[cite:1][cite:6] It shall generate ATS-style observations, weak bullet detection, missing keywords, and improvement suggestions.[cite:1]

Semantic Search and RAG

Semantic retrieval will ship in a narrow, user-private form. It shall work only across the user's own uploaded documents and notes.[cite:1][cite:6] Answers should indicate the underlying source record so users can trace where information came from.[cite:1]

AI Strategy Finalization

Primary AI Provider

OpenAI is the final primary provider for MVP.[cite:1] This decision is frozen for implementation consistency.

Why This Choice

The original brief allowed multiple providers, but a solo build should not start with a multi-provider production matrix because that increases surface area, testing burden, and prompt drift risk.[cite:1][cite:11][cite:14] Provider abstraction can still exist at the service layer, but only one provider should be active in MVP.[cite:11]

AI Workflows That Ship in MVP

- AI assistant chat.
- AI daily brief.
- AI resume review.
- Semantic answer generation over notes and docs.
- Simple recommendation messages on dashboard.[cite:1][cite:6]

AI Workflows Deferred

- AI mock interviews.[cite:1]
- AI project repository deep review.[cite:1][cite:6]
- Specialized multi-agent system.[cite:1][cite:6]
- Company-specific application optimization beyond basic role matching.[cite:1][cite:6]

Data Model Finalization

Final Core Entities for MVP

- User
- UserSettings
- Project
- Task
- Milestone
- Goal
- Habit
- Note
- NoteFolder
- Tag
- LearningResource
- Resume
- JobApplication
- Company
- CalendarEvent
- GitHubAccount
- Repository
- Document
- DocumentChunk

- Embedding
- AIConversation
- AIMessage
- DailyBrief
- Notification
- ActivityLog[cite:1]

Removed from MVP Entity Scope

The following are not required as first-class MVP entities:

- PullRequest
- Issue
- CertificateAuthority-style admin governance models
- Shared workspace membership
- Agent registry[cite:1][cite:6]

Storage Decisions

- All business records are stored in PostgreSQL.
- All uploaded files are stored in Supabase Storage.
- Embeddings are stored in PostgreSQL using pgvector.
- No secondary search engine such as Elasticsearch is used in MVP.[cite:1][cite:11]

Job Processing Decision

All long-running workflows shall use one internal async job pattern. Examples include resume review, GitHub sync, document ingestion, and daily brief generation.[cite:6][cite:11]

Final Implementation Pattern

- API request creates a job row in database.
- Background scheduler polls pending jobs.
- Worker service processes the job.
- Result is written back to domain tables.
- UI polls job status or refetches generated output.[cite:11][cite:14]

This avoids adding Kafka, RabbitMQ, or separate workers too early while still supporting production-safe async execution for a solo-maintained product.[cite:11][cite:14]

Security Finalization

Required Security Controls

- BCrypt or equivalent secure password hashing.
- JWT auth with refresh token flow.
- OAuth2 for GitHub integration.
- Role-based access control.
- Server-side validation using DTO validation rules.
- File upload validation by type and size.
- HTTPS in production.
- Rate limiting on auth and AI endpoints.
- SQL injection and XSS protections through framework defaults and output handling.[cite:1] [cite:15]

Privacy Rules

- User documents are private by default.
- User data must be isolated in every query path, including semantic retrieval paths.[cite:6]
- AI context must never include another user's data.[cite:6]

Non-Functional Requirements

Performance

- Typical non-AI API p95 under 200 ms.[cite:1]
- Dashboard initial load should feel responsive under normal use.[cite:15]
- Semantic retrieval and AI generation can be slower but must expose loading state and job status.[cite:6]

Reliability

- Background jobs must be retryable.
- Failed sync or AI jobs must store error state.
- The app must not crash due to missing optional integrations.[cite:6][cite:15]

Maintainability

- Backend modules follow consistent structure.
- Frontend features follow consistent structure.
- Flyway handles all schema evolution.
- Shared conventions must be frozen before core build begins.[cite:11][cite:15]

Scalability

The MVP should be designed to scale vertically and modularly before any service splitting. Database indexes, caching only where justified, and clean module boundaries are the preferred scalability tools for v1.[cite:11][cite:15]

API Design Rules

The API must follow resource-oriented design for CRUD operations and action-oriented endpoints for AI or sync workflows.[cite:11][cite:7]

Required Endpoint Groups

- `/api/auth`
- `/api/users`
- `/api/dashboard`
- `/api/projects`
- `/api/tasks`
- `/api/goals`
- `/api/notes`
- `/api/learning`
- `/api/resumes`
- `/api/careers`
- `/api/calendar`
- `/api/integrations/github`
- `/api/knowledge`
- `/api/ai`
- `/api/jobs`[cite:11][cite:7]

API Behavior Rules

- Validation errors must be structured.
- Long-running tasks return job IDs.
- Filtering, pagination, and sorting conventions must be consistent.
- Protected endpoints require user ownership checks.[cite:11][cite:7]

Frontend Experience Rules

Product Characteristics

- Responsive design required.[cite:1]
- Dark/light theme support required.[cite:1]
- Accessible UI required.[cite:1]
- Clear empty states required.[cite:6]
- Fast navigation and low-friction forms required.[cite:11]

UX Priority Order

1. Clarity.
2. Cohesion.
3. Speed.
4. Assistive intelligence.
5. Visual polish.[cite:6][cite:11]

Final Roadmap

Phase 0 — Freeze and Scaffold

- Finalize PRD.
- Finalize schema draft.
- Finalize repo structure.
- Create frontend and backend shells.
- Configure auth basics.
- Connect database.
- Set up storage config.
- Add base UI shell.[cite:11][cite:10]

Phase 1 — Core Workspace

- Auth.
- Profile.
- Dashboard shell.
- Projects.
- Tasks.
- Milestones.
- Goals.
- Habits.

- Notes.[cite:1][cite:11]

Phase 2 — Growth Modules

- Learning tracker.
- Resume manager.
- Job tracker.
- Calendar.[cite:1]

Phase 3 — Intelligence Layer

- AI assistant.
- AI daily brief.
- AI resume review.
- Basic dashboard recommendations.[cite:1][cite:6]

Phase 4 — Connected Context

- GitHub integration.
- Document ingestion.
- Embeddings.
- Semantic search.[cite:1]

Phase 5 — Hardening

- Performance tuning.
- Security review.
- Retry logic.
- Observability.
- Production QA.
- Deployment polish.[cite:1][cite:15]

Build Order Decision

Because this is a solo build, development must follow dependency order instead of excitement order.[cite:11][cite:14] The exact build order is:

1. Backend and frontend scaffolding.
2. Auth and protected routes.
3. Database schema and migrations.
4. Projects and tasks.
5. Goals and notes.

6. Dashboard composition.
7. Learning, resumes, careers, calendar.
8. File upload pipeline.
9. AI assistant basics.
10. Resume review job flow.
11. GitHub sync.
12. Semantic search.
13. Hardening and deployment.[cite:11][cite:14][cite:10]

Repo Strategy

Even as a solo developer, the repo should stay professional and predictable. The preferred strategy is one repository containing separate frontend and backend applications with a shared docs area, because that keeps setup and versioning easier to manage while preserving boundaries.[cite:11][cite:7]

Final Repo Shape

- `/frontend`
- `/backend`
- `/docs`
- `/infra`[cite:11][cite:7]

Definition of Done for MVP

The MVP is done only when all in-scope modules work with stable authentication, persistent storage, production-safe file handling, job execution for long-running tasks, and successful deployment in a real hosted environment.[cite:1][cite:15] It is not enough for features to exist locally; they must work as an integrated, user-safe system.[cite:15]

Immediate Next Documents Required

The product blueprint is clear enough to begin implementation planning, but the following build documents should be created next for exact execution:

1. Database schema and ER design.
2. Backend package tree and initial class map.
3. Frontend route map and page inventory.
4. API contract draft.
5. Environment variable matrix.
6. Sprint 0 setup checklist.[cite:10][cite:11]

Final Statement

This PRD freezes the product direction, MVP scope, architecture style, technology stack, AI strategy, and delivery sequence for DevHub as a solo-built production web application.[cite:1] [cite:6][cite:11] Development should begin only against this narrowed and finalized blueprint so that implementation remains focused, maintainable, and shippable by one developer.[cite:11] [cite:14]